Instructions reminder

You must deliver for each of the parts the following artifacts:
1. A PDF report, uploaded to Aula Global in a section that will be available for that purpose. Name the report file like IRWA-2026-uXXXXXX-uXXXXXX-uXXXXXX-part-N.pdf where uXXXXXX is each student's ID. The report must include: an explanation of the decisions you made for
implementing the different algorithms, the assumptions made, and everything else you consider relevant
to explain to the evaluators.
2. The repository TAG that names the part that you are delivering, i.e., IRWA-2025-part-N. The
TAG creation date must be before the delivery deadline in order to be considered. You can find here a
guide on how to work with Git. Section 2.6 addresses tagging.

The team will be evaluated based on the code, corresponding results, the quality of the README.md
explaining how to run the code, and the comprehensibility of the reports.
Your code will be evaluated mainly on the implemented algorithms and the level of reproducibility.
Each part is worth 1 point (4 total)

AI Usage Policy:
If AI tools are used, you must acknowledge this in your submission (e.g., “I used ChatGPT to review my draft explanation of the pre-processing decisions, and debug the stemming function”). You remain responsible for verifying the accuracy, originality, and appropriateness of all submitted work. Misuse of
AI tools may be considered plagiarism or academic dishonesty.

Permitted Uses:
1. You may use AI to review drafts or check grammar and formatting.
2. You may use AI to brainstorm approaches to tasks (e.g., data pre-processing strategies) but not to generate final answers verbatim.
3. AI tools may assist in coding or debugging, but you must understand and be able to explain any code
you submit.
Prohibited Uses:
1. Copy-pasting AI-generated answers as your own without critical review or modification.
2. Using AI to bypass the thinking process required in reflective or analytical tasks.

Things to do before Friday 23rd of October

Marina
Ramon
Shaoya

Part 1

- In **part 1**, create 2 dataframes:
    - *df* has product information without preprocessing (e.g 'description' = 'Wool Black pants made out of 100% cotton')
    - *index_info* only contains the fields we'll use in the index. all fields are pre-processed (e.g 'description' = 'wool black pant mad out 100 cotton')
- explore which 'product_details' subfields might be interesting to include in the index. look inside the json files.
- in the dataframe replace "dirty" numerical data with "clean" version

Part 2

- Missing rate on each field statistics --> We need to process the missing values
- Are the categories balanced? --> Explore top and low categories
    - category and subcategory treemap
- Distribution of price, ratings, discount
- Top sellers and brands regarding product quantities
- % of items in and out of stock
- Distribution of length of title, description, final tokens
- Most common words --> This can be used for TF
- Do a wordcloud with most frequent words in titles and descriptions (?)
- Apply NER to the descriptions:
    - most frequent words of each category (?)
    - try to do a wordcloud for each category

Other stuff

- multicolor write the report :"""(→ each person writes their own part
- document functions and add more explanations
- write the README.md : in the main one we say there's one for each part
    - for this lab, write *progress/part1/README.md*
- when we do the last commit, add a **TAG named IRWA-2025-part-1**

⚠️*The report must include: an explanation of the decisions you made for implementing the different algorithms, the assumptions made, and everything else you consider relevant to explain to the evaluators.*⚠️

Title

Git tag

AI Use Declaration

- claude sonnet 5.5 → revise draft structure

Part 1: Preprocessing the documents

We decided to use `pandas` to preprocess the documents, as is the library we are most familiar with. Our workflow has four main steps:

1. Read the data stored in the `json` file into a `DataFrame`
2. Clean the data: inspect missing values and convert data types
3. Process the text by stemming, tokenizing, etc.
4. Save the results to `csv` files

The sections below explain steps 2 and 3 in more detail and show the reasoning behind our choices in each step.

Step 2. Data quality checks and converting data types

After loading the data, we verify that `pid` exists and is unique for every product. The evaluation labels refer to products by `pid`, so it must identify each product without ambiguity. We then count the missing values in each field, treating empty and whitespace-only strings as missing. Figure 1 shows the result.

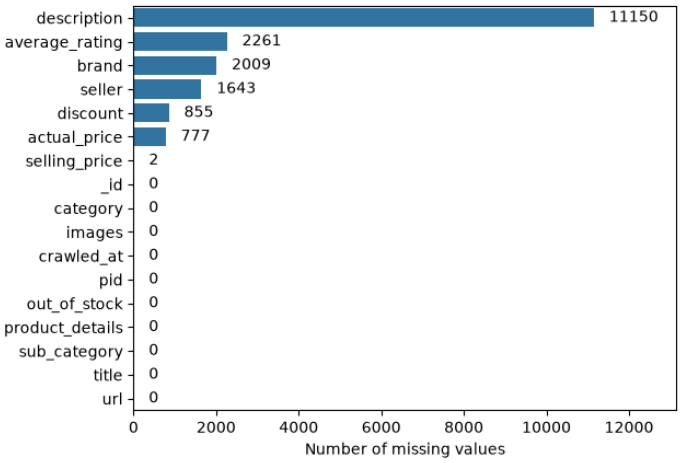


Figure 1. Number of missing values on each field.

We weren't explicitly asked to do this, but it gives us important context about the fashion products dataset — a large percentage of products have no description (40%), average rating (8%), brand (7%), or seller (5%). However, all products have a `pid`, title, category, and subcategory.

Next, we convert all numeric columns (prices, discounts, and ratings) from strings to floats. The column `out_of_stock` is already boolean, so we keep it unchanged. This step is indispensable multiple reasons. First, we need numeric values for the EDA later on — we cannot, for instance, find the average price of all products if they are stored as a string of text. Second, we'd lose functionality in the search engine: the user wouldn't be able to sort the search results by price, or find those with a discount.

Step 3. Text processing

In step 3, we create two tables: `df` and `index_df`. The former, `df`, holds all of the products' information without any text processing. The product's title, description, etc. are kept as they were originally written. For example, in the description of a product in `df` might read: 'Wool Black pants made out of 100% cotton'.

	pid	title	description	category	actual_price	brand	url	out_of_stock
0	TKPFCZ9EA7H5FYZH	Solid Women Multicolor Track Pants	Yorker trackpants made from 100% rich combed c...	Clothing and Accessories	2999.0	York	https://www.flipkart.com/yorker-solid-men-mult...	False
1	TKPFCZ9EJZV2UVRZ	Solid Men Blue Track Pants	Yorker trackpants made from 100% rich combed c...	Clothing and Accessories	1499.0	York	https://www.flipkart.com/yorker-solid-men-blue...	False
2	TKPFCZ9EHFCY5Z4Y	Solid Men Multicolor Track Pants	Yorker trackpants made from 100% rich combed c...	Clothing and Accessories	2999.0	York	https://www.flipkart.com/yorker-solid-men-mult...	False
3	TKPFCZ9ESZZ7YWEF	Solid Women Multicolor Track Pants	Yorker trackpants made from 100% rich combed c...	Clothing and Accessories	2999.0	York	https://www.flipkart.com/yorker-solid-men-mult...	False
4	TKPFCZ9EVXKBSUD7	Solid Women Brown, Grey Track Pants	Yorker trackpants made from 100% rich combed c...	Clothing and Accessories	2999.0	York	https://www.flipkart.com/yorker-solid-men-brow...	False

Figure 2. First five rows of `df`. Not all fields are shown due to lack of space.

The latter, `index_df`, only contains the information needed to build the index. This includes the fields "title", "description", "brand", "category", "sub_category", and"seller" — things a user could type into the search bar if they wanted to find a specific piece of clothing.

Inspecting the `json` file, we noticed `product_details` contains relevant information such as a product's color, pattern or material. Therefore, we decided to include it in `index_df`. This field is a list of single-entry dictionaries, for example {'Fabric': 'Cotton Blend'}. We keep only the values. The keys (Fabric, Color, Pattern) repeat across many products, and they would only add noise if we kept them as terms.

	pid	title	description	brand	category	sub_category	seller	product_details
0	TKPFCZ9EA7H5FYZH	[solid, women, multicolor, track, pant]	[yorker, trackpant, made, 100, rich, comb, cot...	[york]	[cloth, accessori]	[bottomwear]	[shyam, enterpris]	[1005combo2, elast, side, pocket, cotton, blen...
1	TKPFCZ9EJZV2UVRZ	[solid, men, blue, track, pant]	[yorker, trackpant, made, 100, rich, comb, cot...	[york]	[cloth, accessori]	[bottomwear]	[shyam, enterpris]	[1005blue, drawstr, elast, side, pocket, cotto...
2	TKPFCZ9EHFCY5Z4Y	[solid, men, multicolor, track, pant]	[yorker, trackpant, made, 100, rich, comb, cot...	[york]	[cloth, accessori]	[bottomwear]	[shyam, enterpris]	[1005combo4, elast, side, pocket, cotton, blen...
3	TKPFCZ9ESZZ7YWEF	[solid, women, multicolor, track, pant]	[yorker, trackpant, made, 100, rich, comb, cot...	[york]	[cloth, accessori]	[bottomwear]	[shyam, enterpris]	[1005combo3, elast, side, pocket, cotton, blen...
4	TKPFCZ9EVXKBSUD7	[solid, women, brown, grey, track, pant]	[yorker, trackpant, made, 100, rich, comb, cot...	[york]	[cloth, accessori]	[bottomwear]	[shyam, enterpris]	[1005combo1, drawstr, elast, side, pocket, cot...

Figure 3. First five rows of `index_df`.

All fields in are `index_df` pre-processed using the function `build_terms()`. This function performs four operations in this order:

1. Lowercase the text.
2. Tokenize with the regular expression `\w+`, which keeps words and numbers and drops punctuation.
3. Remove stop words using the English stop-word list from `nltk`.
4. Stem each token with the Porter stemmer.

Table 1 shows an example of how it transforms the text.

Before	Yorker trackpants made from 100% rich combed cotton giving it a rich look.Designed for Comfort,Skin friendly fabric,itch-free waistband & great for all year round use Proudly made in India
After	['yorker', 'trackpant', 'made', '100', 'rich', 'comb', 'cotton', 'give', 'rich', 'look', 'design', 'comfort', 'skin', 'friendli', 'fabric', 'itch', 'free', 'waistband', 'great', 'year', 'round', 'use', 'proudli', 'made', 'india']

Table 1. Description of the first item in the dataset before and after applying `build_terms()`.

We assume queries will go through the same `build_terms()` function as the documents, otherwise stemmed document terms would not match raw query terms.

Justification

We use two dataframes to separate the data shown to the user from the data needed to build the index. `df` keeps the original text because stemmed terms are unreadable in a results list (`friendli` instead of “friendly”). `index_df` stores the preprocessed terms, so the next part of the project can build the index without repeating the preprocessing. It is also the table we use for the exploratory data analysis (EDA). Both tables share `pid`, which links every index entry to the full product information.

For the text fields, we compared two options: merging them into a single string, or keeping them as separate fields. In the end, we chose to keep them separate. The first option is the

simplest, but it's less flexible. If we merge all fields, a match in the title would count the same as a match in the seller name.
The second option would let us give each field its own weight, which we could tune with `validation_labels.csv` in the next part of the project. The disadvantage of separate fields is more complexity: scoring must combine multiple fields. It also makes the index larger and harder to build. However, space is not the real problem, because the dataset is small (~60 MiB in json format, ~45 MiB in csv). We accept the extra complexity of this option because we think it'll lead to better performance in the future.

Finally, we store both tables as `csv`. It is the natural format for a dataframe, and we can easily convert it back to `json` if needed. It has one limitation, though: list columns (`product_details` and the term lists in `index_df`) are written as strings, so they must be parsed correctly when the files are loaded.

Part 2: EDA

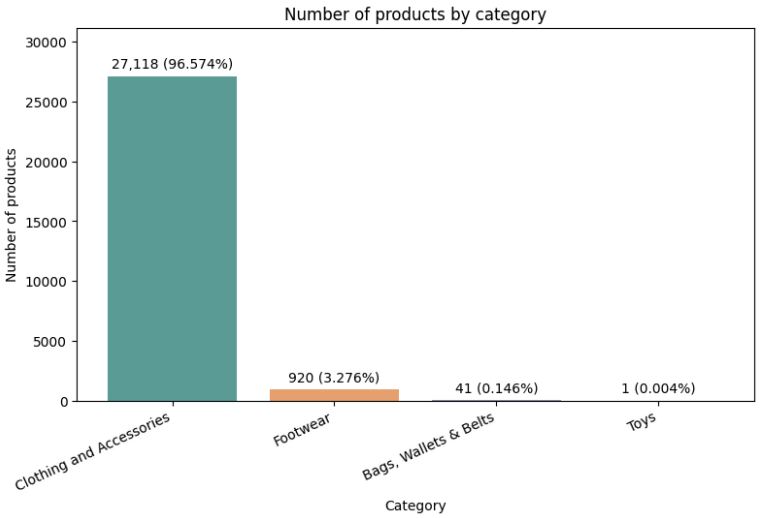
Feel free to do the exploratory analysis and report your findings in the report.

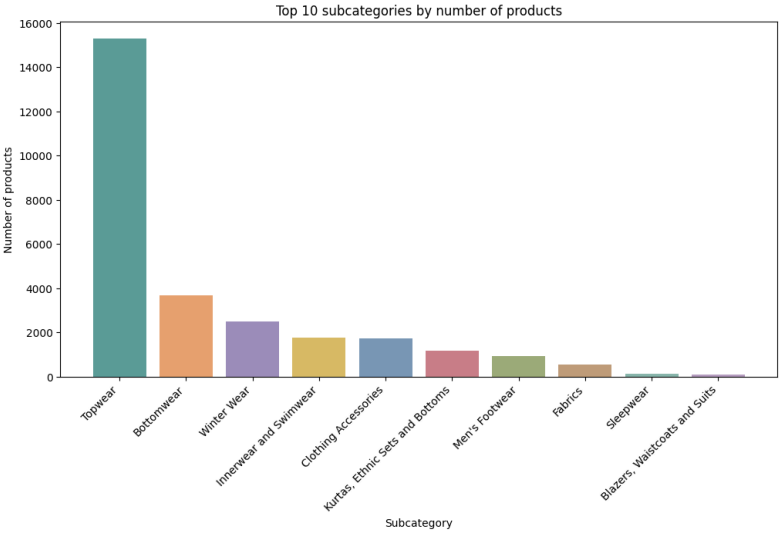
- Why do we need to do an analysis on the fashion product dataset, how it helps us to make decisions
- why do we explore the degree of balance among categories
- why distribution of price, ratings and discount
- why Top sellers and brands based on product quantities
- why % of items in and out of stock

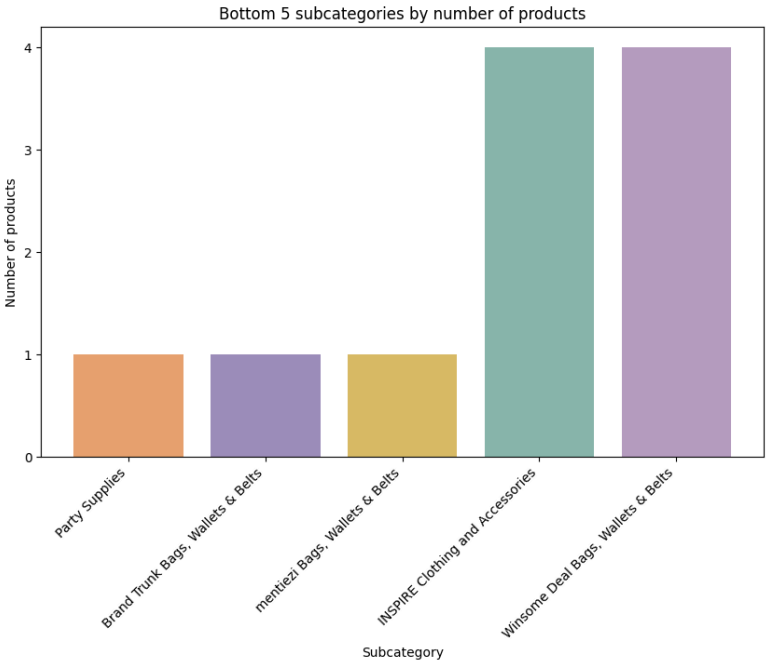
Data analysis is fundamental and essential for us to better understand how the products are formed, how their numerical features will be and their status of stocks in the dataset. All these analyses help with the data distribution, missing values, and among others, which set a solid foundation for our future indexing design.

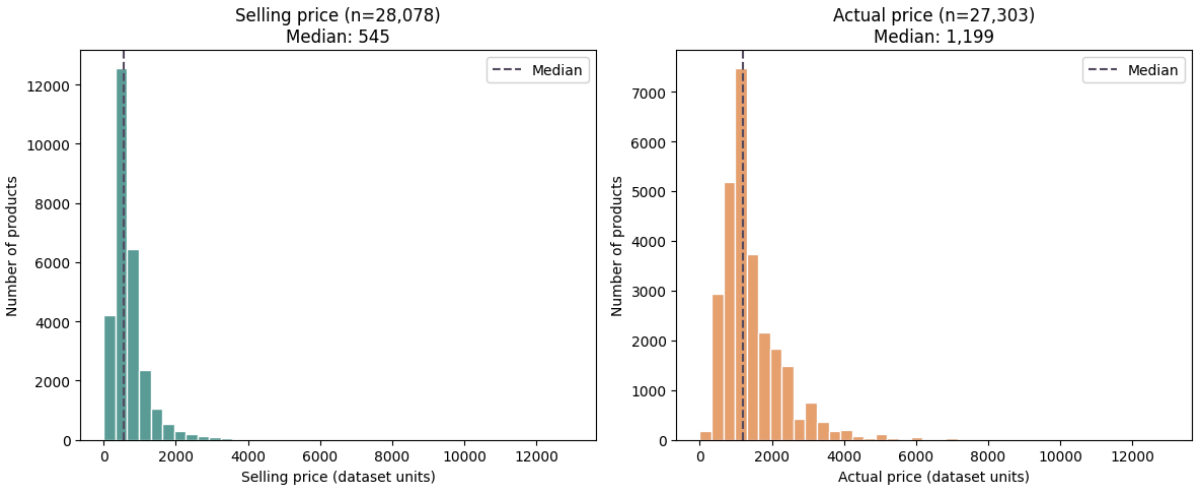
Based on the characteristic of the dataset provided, we consider useful to explore the features such as the balance among categories, distribution of price, ratings and discount of the products, top sellers and brands with respect to the product quantities and also rate of items in and out of stock

1. Balance of categories and subcategories

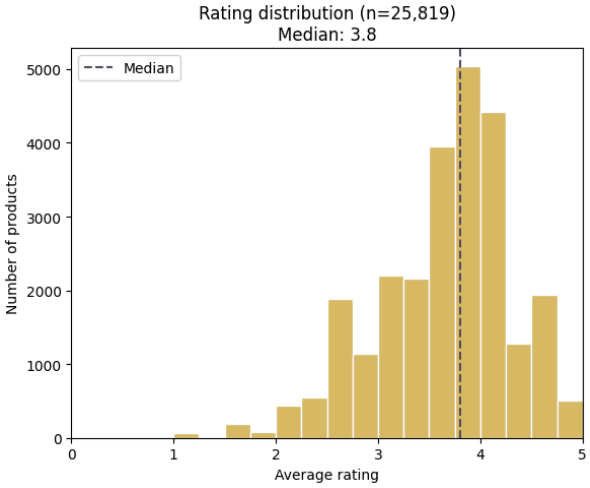

Number of products by category
30000
25000
20000
15000
10000
5000
0
Number of products
27,118 (96.574%)
920 (3.276%)
41 (0.146%)
1 (0.004%)
Clothing and Accessories
Footwear
Bags, Wallets & Belts
Toys
Category

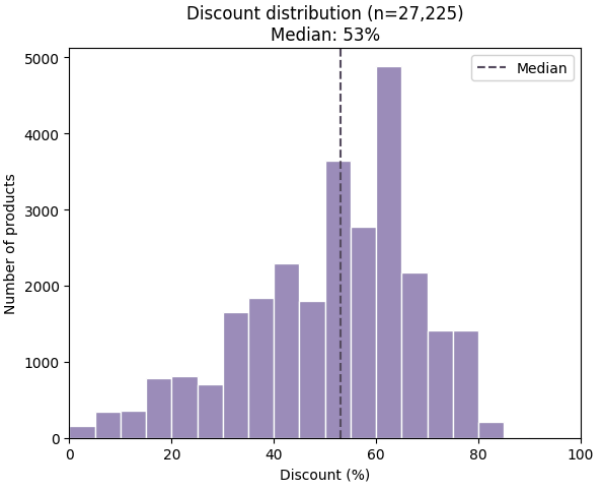

Top 10 subcategories by number of products
16000
14000
12000
10000
8000
6000
4000
2000
0
Number of products
Topwear
Bottomwear
Winter Wear
Innerwear and Swimwear
Clothing Accessories
Kurtas, Ethnic Sets and Bottoms
Men's Footwear
Fabrics
Sleepwear
Blazers, Waistcoats and Suits
Subcategory

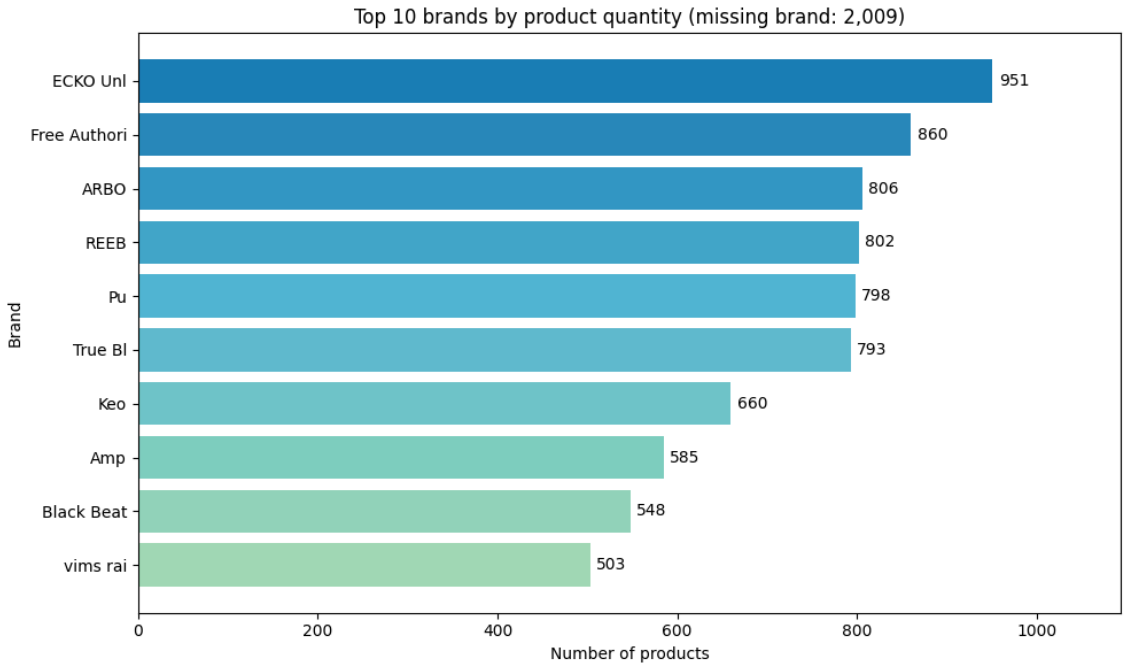

Bottom 5 subcategories by number of products
4
3
2
1
0
Number of products
Party Supplies
Brand Trunk Bags, Wallets & Belts
mentiezi Bags, Wallets & Belts
INSPIRE Clothing and Accessories
Winsome Deal Bags, Wallets & Belts
Subcategory


Selling price (n=28,078)
Median: 545
Median
Number of products
Selling price (dataset units)
0
2000
4000
6000
8000
10000
12000

Actual price (n=27,303)
Median: 1,199
Median
Number of products
Actual price (dataset units)
0
1000
2000
3000
4000
5000
6000
7000
0
2000
4000
6000
8000
10000
12000


Rating distribution (n=25,819)
Median: 3.8
Median
Number of products
Average rating
0
1000
2000
3000
4000
5000
0
1
2
3
4
5


Discount distribution (n=27,225)
Median: 53%
Median
Number of products
Discount (%)
0
1000
2000
3000
4000
5000
0
20
40
60
80
100


Top 10 brands by product quantity (missing brand: 2,009)
Brand
ECKO Unl
Free Authori
ARBO
REEB
Pu
True Bl
Keo
Amp
Black Beat
vims rai
951
860
806
802
798
793
660
585
548
503
0
200
400
600
800
1000
Number of products